

Report ITPE3200 – Web Applications

Group: 5478

Members:

199

271

249

135

Table of contents

Table of contents.....	1
Introduction	2
Summary of the Web Application:.....	2
Description of Target Users:	2
Versions of Node.js, ASP.NET, React that are used:	3
Running the project	3
Create account and login.....	3
How ChatGPT is used:.....	4
Who contributed to what?.....	5
Requirements Analysis	6

Why people need our web application:	6
Required functionalities	7
Software Architecture	7
Software architecture of sub-application 1.....	7
Functionality	9
User stories.....	8
Models	9
Controllers.....	11
Views.....	11
Modules/Pages in Sub-Application 2	12
Data Access	13
Chose database technology	13
Database structure.....	13
Database connection to the web app	15
Repository pattern design (for sub-application 1).....	15
API service layer design (for sub-application 2).....	16
Unit Tests	16
Unit test implemented:.....	16
Expected results of the Unit Tests:	17
User Experience	18
User interface designed:	18
User experience test:	18
Considerations for Usability:	19
Considerations for Accessibility:	20
Conclusion and Reflection	20

Introduction

Summary of the Web Application:

The project is a web application for students to organize, share and collaborate on notes and materials for their studies.

Description of Target Users:

The main target users for this web application are people who often feel overwhelmed with assignments, projects, or just enjoy sharing their knowledge and creativity through notes and pictures.

This includes students, hobbyists, creative professionals, and anyone looking for an organized platform where they can share ideas, get feedback through comments, and connect with a community of like-minded individuals

Versions of Node.js, ASP.NET, React that are used:

Node.js: v22.11.0

ASP.NET: .NET 8.0

React: 18.3.1

Running the project

Sub application-1 can be run using these commands after opening the project:

- Open new terminal
- Cd WebMVC
- Dotnet ef database update
- Dotnet run
- Open browser
- <http://localhost:5000>

Sub application-2 can be run using these commands after opening the project:

- Backend:
 - Open new terminal
 - cd api
 - Dotnet ef database update
 - dotnet run
- Frontend:
 - Open new terminal
 - cd reactapp
 - Npm install
 - npm start

Create account and login

Create an account:

To create an account, the user can click the Login button located in the top-right corner of the application. This opens the login screen, where they will find an option to register a new account. The user can then sign up by providing their email address and creating a password. After registering the account, the user is sent back to the login screen and can log into their new account.

Login as admin:

To login as an admin the user can use the email and password:

Email: admin@gmail.com

Password: Admin123!

How ChatGPT is used:

We used ChatGPT to help us resolve errors and bugs that occurred during development. An example of this would be in the beginning of development, we had some trouble with merging branches in our GitHub repository. ChatGPT helped us locate an issue in the .gitignore file, which led to unnecessary files being uploaded to our remote repository, complicating the merges. ChatGPT was also used to find the cause of errors and warnings, where it provided step-by-step guidance on how to fix the problem, be it a missing import, an incorrect path in the file structure or a syntax issue.

ChatGPT was utilized to provide explanations and guidance throughout the project development. Early on it, it helped us explore various approaches for achieving desired outcomes using .NET, helping to determine the best practices and solutions for specific feature. ChatGPT also played a key role in implementing features beyond course examples, such as the upvote functionality. It helped design and implement the feature, allowing a user to upvote a post once or remove the upvote if attempted again.

ChatGPT was a great help when we ran into issues during the unit testing phase of the project. For example, the unit test for the Index method, which had been working fine before, suddenly started failing. ChatGPT provided clear explanations and guided me step by step to figure out what went wrong and how to fix it. It helped me look into possible causes, like changes in the data context, problems with the mocking setup, or issues in the method implementation

ChatGPT was used to create mock posts that is injected into the database to populate the database while developing the application. This was useful as it saved a lot of time instead of manually populate the server. It also provided some insight into some of the

weaknesses of our database logic, such as the fact that none of the users (authors) of the mock posts exists in our database. This highlighted the need to establish proper relationships between posts and users to ensure data integrity.

Who contributed to what?

Student 249:

- **Wrote Chapters:** “Software architecture” and “Data Access”.
- **Coded:**
 - Developed Controllers, Repositories and Models for the Post entity.
 - Implemented Upvote/like functionality.
 - Setup of React app, with communication between frontend and backend.
 - Implemented post pages and components in the React app.

Student 271:

- **Wrote Chapters:** “Data Models”, “Main and sub-functionality”, “Running the project” and “Conclusion and reflections”
- **Coded:**
 - **Subapp 1**
 - Developed mainpage with tags, search, extending and folding descriptions
 - Created mypage, for users to view and edit their own posts
 - **Subapp 2**
 - Changed from jsx to tsx
 - Developed homepage, with, search, tag and myposts
 - Bug fixing and testing
 - Simplifying code for readability

Student 135:

- **Wrote Chapters:** “Controllers” and “Modules/Pages in Sub-Application 2”
- **Coded:**
 - **Subapp 1:**

- Developed the entire comment functionality, including adding, editing, and deleting comments.
- Implemented back-end logic for managing comments through the CommentController.
- Created database interactions for comments using repositories to ensure smooth CRUD operations.
- Integrated comment-related functionality into the MainPage, ensuring seamless user interaction.
- **Subapp 2:**
 - Developed the API endpoints for comments in the .NET backend.
 - Designed and implemented the React components for the comment section within posts.
 - Ensured AJAX communication between the frontend and backend for comment functionality.
 - Managed testing and bug fixes for all related features for comments.
 - Simplified and optimized the codebase for readability and maintainability.

Student 199:

- **Wrote chapters:** “Functionality”, “Unit Tests”, “User Experience”
- **Coded:**
 - Unit testing
 - AdminDashController
 - logging with serilog
 - add authorization through scaffolding in both Sub-application 1 and Sub-application 2, connecting it to the rest of the application
 - Made LoginModalComponent in sub 2
 - Implemented roles and an automated creation of an admin user
 - error handling
 - user interface design, user experience design, user test part of the application.

Requirements Analysis

Why people need our web application:

Students often struggle to find an organized and efficient way to collaborate and share information about their subjects and assignments. While many currently use platforms like Discord, these solutions lack features tailored specifically to academic needs. This

web application addresses that gap by providing a dedicated platform for students to share posts about their studies, discuss assignments, exchange lecture notes, and engage with other school-related topics. By mixing the functionality of social media and forums, this application aims to offer an intuitive and structured application for academic collaboration, as well as a sense of community.

Required functionalities

For this application the required functionalities are explained by user stories:

- The users should be able to create a post and add images if needed. This is important as it is the main feature of this application.
- The users should be able to edit and delete their own posts. This is important in case the user wishes to change their posts or delete it altogether.
- The users should be able to upvote a post. This is important to create a sense of a community and to highlight the most popular posts in the application.
- The user should be able to comment on a post. This is important as it facilitates discussions, promotes interaction among users, and enhances engagement within the application.
- The user should be able to add tags to a post and search posts by tags. This is important as it helps users navigate the application and quickly find posts related to specific topics or subjects.
- The user should be able to sort posts, based on some parameters (in our case old and newest posts).
- The users should be able to create an account and log in with it. This functionality is crucial as it allows users to manage their personal content, such as editing and deleting their own posts. It also enables users to see who has commented on their posts, creating a sense of a community and engagement.
- The admin should have access to their own dedicated Admin Dashboard. This provides an interface for efficiently managing the application's posts and comments.

Software Architecture

Software architecture of sub-application 1

The software architecture for sub-application 1 uses the Model-View-Controller (MVC) pattern.

- **Model:** Represents the data and business logic, containing entities like Comment, Likes, and Post. The Model uses the Data Access Layer (DAL) for database interactions.
- **DAL:** Manages database operations through CRUD actions. It includes repositories for each Model that handle the access and modification of data in the database.
- **View:** Handles the user interface (UI), displaying data to the user and capturing user input. Views are organized under sections like AdminDash, Main, and Post. Shared elements, such as _Layout.cshtml, provide a consistent layout for the application.
- **Controller:** Acts as an intermediary, handling user input from the View, interacting with the Model for data manipulation, and determining which View to render.
- **ViewModels:** Is a bridge between the model and the View, helping to format and prepare data display in the View.

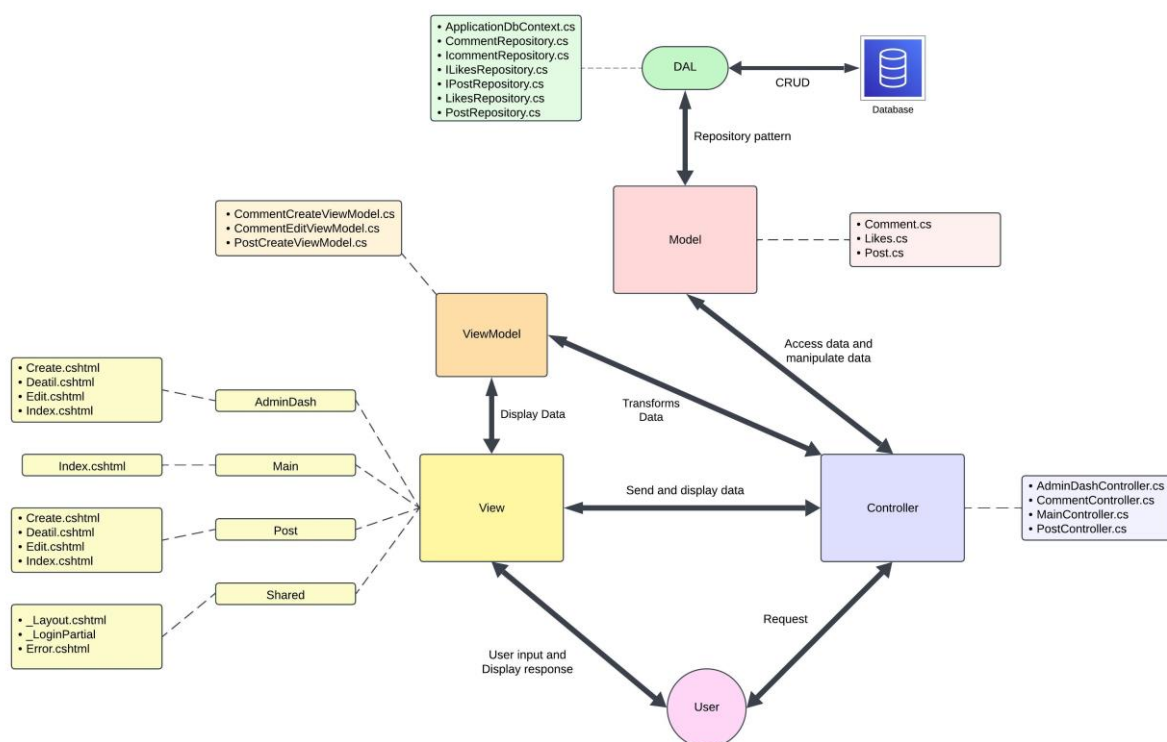


Figure 1: A visual representation of the software architecture in sub-application 1, showing the Model-View-Controller (MVC) pattern and interactions.

Software architecture sub-application 2

The software architecture of sub-application 2 follows a client-server pattern, with a React frontend, an API Layer and a .NET backend.

- **Client (Frontend):** The client is built using React, which handles the user interface, displaying data, and capturing user input. Our frontend app consists of three main pages (home, post, and admindashboard), each using the shared component NavMeny.tsx and public for images. These pages contain different pages, such as PostListPage.tsx, which in turn renders various components (PostList.tsx, PostGrid.tsx, MyPost.tsx) based on user input.
- **API Layer:** The API Layer serves as an intermediary between the frontend and backend, processing HTTP requests and responses.
- **API Controllers (Backend):** The API controllers handle the HTTP requests from the client, apply business logic, and send appropriate responses back to the client.
- **DAL (Data Access Layer):** The DAL manages database operations for accessing and modifying data. It acts as a separation between the data access logic, and the business logic.
- **Models:** The Models represent the data and core business logic, creating the entities and mapping to the database tables. The entities created here are used by the DAL to interact with the database.
- **DTOs/ViewModels:** The DTOs/ViewModels transform data between the client and server, ensuring that only relevant information is sent and in the right format.

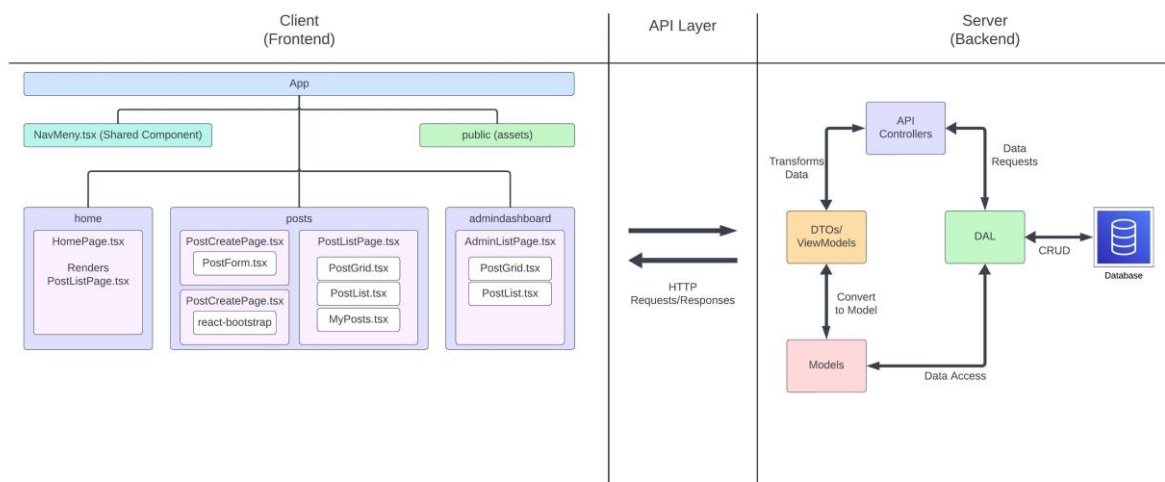


Figure 2: A visual representation of the software architecture in sub-application 2, showing the separation between the Client (frontend using React), API Layer, and Server (backend using .NET).

Functionality

Models

Models in MVC architecture handle the application's data and business logic. They define the structure of the data, manage its relationships, and ensure everything follows the necessary rules and validations. Models interact with the database, making it easy to retrieve, update, or save information. By keeping data logic separate from the user interface and controller, models make the application cleaner, easier to maintain, and more reliable, acting as the foundation for data management.

Comment.cs:

- Represents user comments on posts.
- Id: Unique identifier for each comment.
- Text: The content of the comment, with validation for maximum length and requirement.
- Author: The user who wrote the comment.
- PostId and Post: Link to the associated Post.
- CreatedDate and LastModifiedDate: Timestamps for tracking comment creation and updates.

Likes.cs:

- Represents likes for posts.
- LikesId: Unique identifier for a like entry.
- IsLike: Boolean indicating whether it's a "like" or a "dislike."
- PostId and Post: Nullable references to the associated post.
- UserId and User: Nullable references to the user who liked/disliked the post.

Post.cs:

- Represents posts content.
- Id: Unique identifier for each post.
- Title and Content: The main content of the post, with validation for required fields and length.
- ImageUrl: Optional URL for an image associated with the post.
- Author: The author of the post.
- Comments: Collection of associated Comment objects.
- Upvotes: The number of upvotes the post has received.
- Tag: Optional tag to associate a post with a theme
- Additional fields for timestamps, and a calculated CommentCount.

ErrorViewModel.cs:

- Handles error information for the application.
- RequestId: A unique identifier for the request that encountered an error.

- ShowRequestId: A boolean property to determine whether the RequestId should be displayed (e.g., for debugging).

Controllers

In this project, controllers serve as a bridge between the user interface and its logic, managing interactions between the users, models, and views. Each controller has a given role, designed to facilitate functionality and enhance the modularity of the overall application.

The “AdminDashController.cs” is responsible for handling administrative tasks, such as editing, deleting, and revealing comments and posts. This controller is crucial for maintaining the integrity of the platform, ensuring administrators can have an overview and manage content. On the other hand, “AuthController.cs” is used exclusively in Sub-Application 2 and manages user authentication processes, which includes: login, registration, and session handling. The main purpose of it is to ensure secure access control and provide users with a safe platform to interact with.

The “MainController.cs” present only in Sub-Application 1, acts as a central area for browsing and interacting with posts. It has features like filtering posts by tags, upvoting, and displaying comments, offering a streamlined user experience. The “CommentController.cs” handles the management of comments, allowing users to create, edit, and delete their chosen inputs. This contributes a higher form of user engagement which again enhances the interactive aspect of the application.

For post management, the “PostController.cs” or “PostApiController.cs” in Sub-Application 2, integrates the CRUD operations for posts. They are the core element of the platform. This dedicated controller ensures efficient and organized handling of related tasks with posts. Finally, the “UpvoteController.cs” put into the “MainController.cs” in Sub-Application 1, manages the upvote functionality. It allows users to highlight popular posts by adding or removing upvotes. This also allows the user to create more engagement.

By having these responsibilities to specific controllers, the application ensures a distinct separation of functions. This approach enhances the modularity of the system, simplifies the maintenance and improves the overall user experience.

Views

AdminDash: is connected to the AdminDashController. The views here are nearly a copy of how they are in the View/Post, just added some code to only allow user that

have the role of admin to do actions on view. This separation makes it clear that these views are specific to administrative tasks.

Main: is connected to the MainController. The Index view here acts as the central hub for displaying all posts with features like filtering by tags, upvoting, and commenting. Separating this from the Post views is important to keep the overall user experience distinct from specific post management tasks.

Post: is connected to the PostController. These views (Create, Details, Edit, and Index) are specifically designed to manage posts. The Create view lets users add new posts, while Edit allows them to update existing ones. Details shows all the information about a single post, and Index provides a list of all posts.

Shared: The views in the Shared folder are used for common elements and functionality that are needed across the whole application. These views are placed in the Shared folder to make them reusable and accessible across the application

_Layout.cshtml: This is the main layout file, setting up the structure for all pages, like the header, footer, and navigation bar.

_LoginPartial.cshtml: A small view for handling login and logout links, making it easy to include authentication features in the layout.

_ValidationScriptsPartial.cshtml: Adds scripts for validating forms on the client side.

Error.cshtml: A view for showing error messages when something goes wrong, helping users understand what happened.

Modules/Pages in Sub-Application 2

Sub-Application 2 incorporates several key modules and pages to ensure a seamless user experience. Each module is designed to fulfill specific functionality and is a combination of .NET backend and React frontend framework. The “Login/Register Page” is essential for managing user access to the platform. It allows users to securely create an account or log in to an existing one. This provides personalized access to features such as creating and interacting with posts. The “Homepage” serves as a central area of the application, displaying a selected feed of posts and offering features like sort and filter. This page is designed to be the user’s primary point of interaction.

The “Create Post Page” allows users to contribute content to the platform. That includes an interface for adding text, images, and optional tags. This enables users to share their knowledge and creativity. An addition to this is the “Post Details Page” which displays all the information about a single post. It includes features like viewing

comments, upvoting, and engaging with the content contributing to better user interaction.

The “Comment Section”, integrated as a module within posts, engages discussions and feedback. It allows users to add, edit, or delete comments, creating community engagement and interaction. Another fundamental feature is the “Tag/Search Module”, which enhances navigation by allowing users to filter posts based on tags. This improves the discoverability of content and ensures users can quickly find the information they are looking for.

The “AdminDashboard” page is dedicated for administrators. It offers tools to manage posts and comments. Therefore, the platform remains organized and free from inappropriate content. Finally, the “NavMenu” acts as a consistent navigation bar across the application which provides easy access to key sections such as homepage, the admin dashboard and user account options. Every module and page are important for the functionality and usability of Sub-Application 2. Which creates a coexistent and interactive platform for both users and administrators.

Data Access

Chose database technology

SQLite was chosen for this project because it is lightweight and simple to integrate with ASP.NET and Entity Framework. Its simple setup makes it ideal for rapid prototyping and smaller projects like ours. The compatibility with Entity Framework also simplifies the management of database migrations and data access through the ApplicationDbContext. It also runs on almost all operating systems, including Windows, macOS, Linux, Android, and iOS, which helps us work on the project across different platforms.

As a relational database, SQLite provides us with the opportunity to add relationships using primary and foreign keys for the different entities in our project. It also offers an effective way to retrieve and manipulate data, as it supports complex SQL queries that allow us to efficiently work with related data across multiple tables.

Database structure

The database consists of 11 total tables, where 7 are generated by ASP NET Identity to handle user authentication and identity management. 3 tables are related to our application: posts, comments and upvotes, which represent the core entities of our system for managing user generated content and interactions. The final table, `_efmigrationhistory`, is used by Entity Framework to track database migrations and maintain version control.

The entity: posts, serves as the core of the application, representing the primary content that users create and share. It acts as the central hub for user interactions.

The other two entities: comments and upvotes, are directly linked to posts through a foreign key (postid) in each their tables. This foreign key establishes a relationship between posts and comments, so that each comment can be associated with the post they belong to. For the upvotes table, it establishes a connection between users and posts, tracking which user has upvoted which post and preventing multiple upvotes from the same user on a single post.

There is no foreign key relationship between the entity posts and the entity user (AspNetUser) in our project. The relationship is handled purely in the application code and not enforced in the database itself. We recognize that this can lead to some potential data integrity issues. An example of this is when a user is deleted, their posts remain. The solution to this in our project is that the administrator has the option manually delete all posts by all users. While this approach is not a scalable solution, we found it to be sufficient for the scope of our current project.

The reason for this missing relationship is that the early setup of the project, which was based demos and guidelines provided course literature, where this relationship never was established. We had come quite far in the development before we became aware of the issue. Due to time constraints, we decided to keep our in-code handling of the relationship between posts and user and focus on other parts of the project. With more time for development, we would have implemented a stronger relationship between the users and their posts.

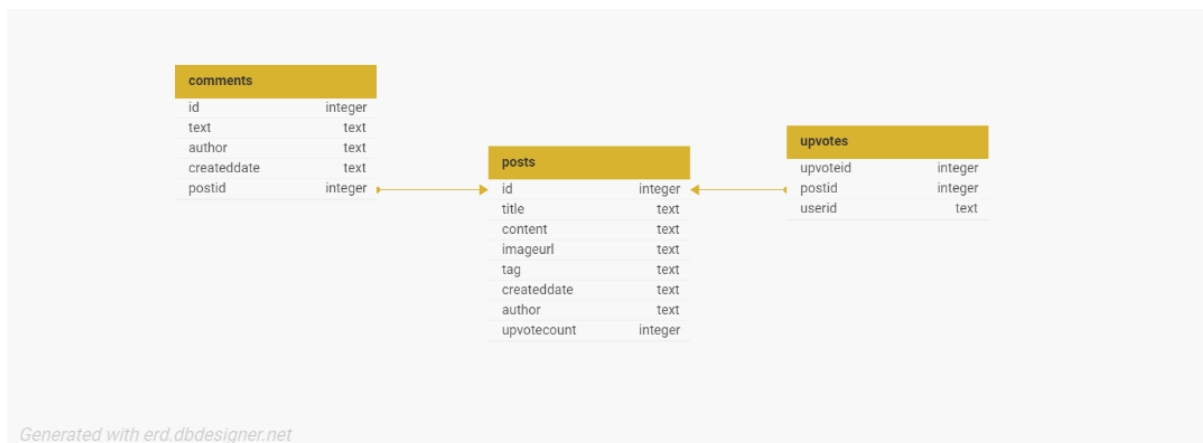


Figure 3: Entity Relationship Diagram for Posts, Comments, and Upvotes in the Web Application

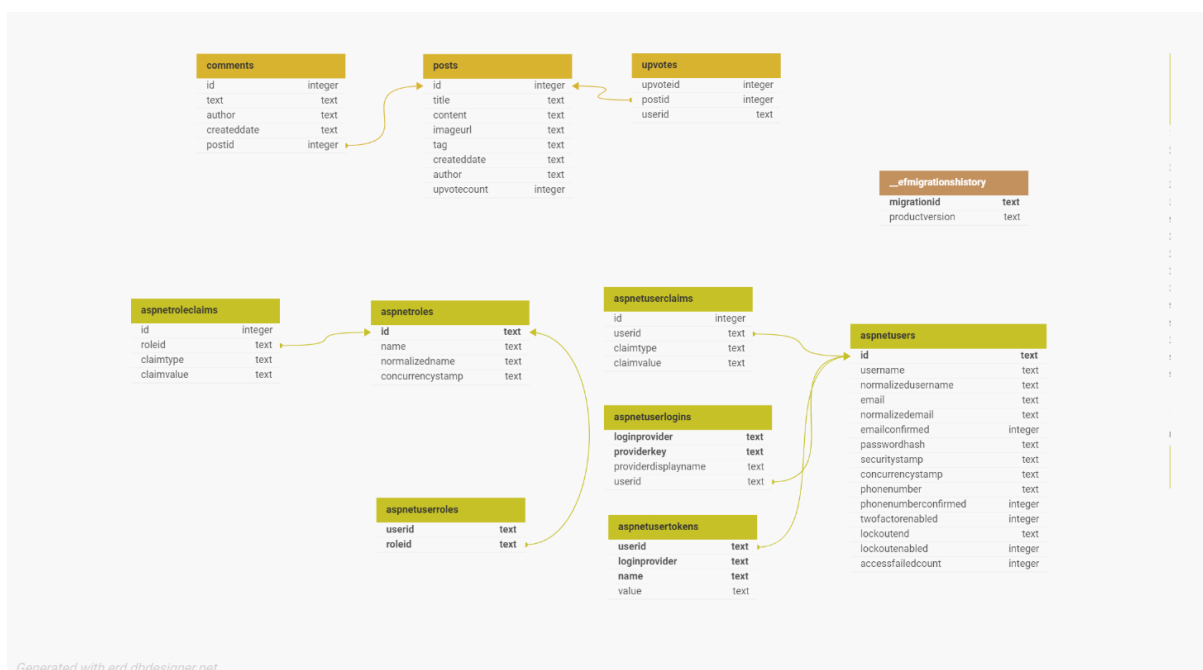


Figure 4: Entity Relationship Diagram for the Web Application Database Schema with ASP.NET Identity.

Database connection to the web app

The database is connected to the web app through the file `ApplicationDbContext.cs`, which serves as a central location for managing data operations. It also inherits the controller class of `IdentityDbContext`, that enables seamless integration with ASP.NET Identity tables to support user authentication and role management.

- **Entities:** The `ApplicationDbContext` manages key entities such as `Post`, `Comment` and `Upvote`, providing the `DbSet<Post>`, `DbSet<Comment>` and

DbSet<Upvote> properties that handles the Create, Read, Update and Delete (CRUD) on these tables in the database.

- **Configuration:** It is configured in the Startup.cs using dependency injection to ensure it is available throughout the entire application when data access is needed. The connection string is specified in the appsettings.json, providing the name of the database.
- **Entity Framework Core:** Entity Framework Core is used as the ORM, or Object-Relation Mapper, to allow us to interact with the database in LINQ in C# instead of using SQL. This simplifies the data access, making the development process easier and reduces the potential for SQL-related errors.

Repository pattern design (for sub-application 1)

The repository pattern design is commonly used in .NET MVC applications to abstract the data access logic. It helps make the application more modular and maintainable, as it acts as a bridge between the applications business logic and the database.

The repository pattern design used in this application consists of three different “pairs” of repository interfaces and repository classes: IPostRepository and PostRepository, ILikesRepository and LikesRepository, ICommentRepository and CommentRepository.

The repository interfaces define the data operations that the repository handles (e.g., GetAll(), GetById(int id) ...). The corresponding repository classes handle the actual database operations defined by the interface, using the Entity Framework to interact with the database.

This repository design ensures that the business logic is handled separately from the data source (the database). By isolating the data access layer from the rest of the application, the code becomes more modular and maintainable.

API service layer design (for sub-application 2)

In sub-application 2 we use an API service layer to act as a bridge between the React frontend and the .NET API backend.

The API service layer is responsible for managing all HTTP requests from the frontend to interact with the backend API. It uses the native fetch API, with the base URL defined in the apiConfig.tsx.

The API service layer also handles the authentication and role management with the use of ASP.NET Identity. This ensures secure data handling, including user login and access control based on the role of the logged in user (user and admin roles in our

project). This ensure that only authorized users can perform certain actions.

Unit Tests

Unit test implemented:

Entities covered:

- **Posts:** AdminDashController and PostController (both tests include CRUD operations)
- **Comments:** CommentController (Test focus on creating, editing and deleting comments associated with posts)

Types of tests:

Positive tests: Validate correct behaviour when inputs and conditions are valid. The positive test we have are:

- Ensuring that a valid post or comment is created successfully
- Verifying authorized actions, such as editing or deleting a post by the author or an administrator
- Checking correct data rendering in the Index and Details views

Negative tests: Test error handling when inputs and conditions are invalid. The negative test we have are:

- Trying to fetch a non-existent post or comment
- Unauthorized actions like editing or deleting a post/comment created by another author
- Handling invalid inputs, such as an empty comment text

Purpose of unit test:

- **Validation:** to make sure the application works as expected in both valid and invalid scenarios
- **Error handling:** to confirm that the system handles unexpected or invalid inputs without crashing
- **Security:** to check only authorized users can perform specific actions like editing or deleting content, ensuring proper access control is in place
- **Reliability:** To verify that the controllers communicate properly with the repositories and return views

Expected results of the Unit Tests:

For Positive Tests:

- Posts and comments should be successfully created, edited, and deleted when valid inputs and authorization are provided.
- Authorized users should be able to access, edit, or delete their posts or comments.
- Correct data should be passed to views (Index, Details, etc.), matching the mock data.

For Negative Tests:

- Attempting to access or modify non-existent posts/comments should return a NotFound result.
- Unauthorized actions should return a Forbid result, preventing unauthorized modifications.
- Invalid data submissions should return BadRequest or display appropriate error messages in the view

User Experience

User interface designed:

The user interface is designed to be simple and clear, focusing on providing a user-friendly and functional experience. Key elements of the design include:

Content display: Posts are displayed as cards, which visually separate each piece of content. This format helps in organizing information clearly and makes each post's details easy to access.

Main function: The "Create New Post" button is placed at the top of the page, encouraging users to contribute content to the platform. It stands out from the other elements due to its central position and distinct colour, drawing the user's attention.

Consistent navigation: A fixed navigation menu at the top provides quick access to key sections like "Home" and "My Posts." This ensures users can easily navigate without confusion.

The logout button and dark mode toggle are conveniently placed.

Functional buttons: Action buttons like "Show Comments" and "Upvotes" are clearly labelled and styled for visibility, encouraging user interaction with posts.

Use of White Space: Generous use of white space around elements keeps the interface uncluttered, enhancing readability and focus on key features

User experience test:

We conducted a user experience test by involving friends and family as testers on application 2. The tests highlighted strengths and areas for improvement.

Strengths:

- The welcome message clearly communicates the purpose of the site, making it clear and inviting for users.
- Navigation is intuitive and well-organized, with links to "Home," "My Posts," and "Admin Dashboard," as well as a visible Logout button for account management.
- The dark mode toggle is a smart feature for users who prefer alternative viewing modes, enhancing accessibility and usability.
- Posts are displayed in a card format, which is visually appealing and easy to scan, providing users with essential details at a glance and works as a divider for different posts.
- The search bar is a valuable addition for filtering content and improving discoverability.

Could be improved:

- The greeting ("Welcome, user@gmail.com") could include the user's name instead of their email for a more personalized experience.
- The background colour feels plain and could more visually appealing.
- Could have more filtering tools for the posts

Reflections:

Testing with friends and family gave us great feedback on the platform's usability. The user experience test showed that the site has a solid foundation with well-designed features and an intuitive layout. However, there's room for improvement like fine-tuning the design and adding more user-focused functionalities such as better personalization options to enhance the overall experience.

Based on the feedback we made a change to the background colour, switching from plain white to a soft parchment yellow, this adds more character. We also added a simple filtering option for the posts, that gives the user the possibility to see the newest post first or the oldest first. Another idea for future improvements could be including the user's name and profile picture to make the platform feel more personal and engaging.

Considerations for Usability:

We have taken several steps to make the site user-friendly, focusing on key aspects of usability.

Intuitive Navigation:

The navigation menu is designed to be clear and easy to use, with links like "Home," "My Posts," and "Admin Dashboard" to help users quickly find what they need. Important actions like logging out and toggling dark mode are prominently displayed and simple to access.

Content Display:

Posts are presented in a card format, which is easy to scan and organizes information clearly. The search bar allows users to filter posts by tags, making it quicker to locate specific content. Same with “newest first” function.

Consistency:

Efforts have been made to keep button placement and design consistent across the site. However, small improvements in spacing and alignment would make the interface feel even more polished.

Responsive Design:

We knew the importance of ensuring the site works well across different devices. Adjustments like resizing cards and buttons for smaller screens are particularly important for mobile users. Our site transitions to a hamburger menu on smaller screens, with buttons and text resizing appropriately. This ensures the content remains readable and the design retains its style and usable.

Error Prevention:

For critical actions like deleting a post the user gets a confirmation pop-up preventing accidental loss of posts.

Considerations for Accessibility:

Dark mode:

We added dark mode to improve accessibility. This feature changes the background and font colours, providing an option that may be easier for some people to read, especially in low-light environments or for those with visual sensitivities.

Colour contrast:

We have focused on colour contrast to make our platform accessible to more users. We have tried to meet the recommended standards in most areas but know there might still be places where we can improve contrast.

Error Feedback:

We have added error feedback to help users when they make mistakes, like entering invalid input. For example, if someone tries to log in without entering an email, the site shows a clear message saying, "Please enter an email." This helps users understand what went wrong and how to fix it

Conclusion and Reflection

This project was a big learning experience for all of us. We got hands-on with building a full-stack web app using .NET, React, and SQLite, which really helped us understand how modern frameworks work together. Troubleshooting issues gave us a chance to dig deep into MVC and React, and we got much better at figuring out what went wrong and how to fix it. On top of that, working as a group showed us how important teamwork is, especially when solving tough problems.

The biggest challenges we faced were connecting the frontend to the backend, managing database relationships, and dealing with GitHub. Making the API work smoothly with React components took a lot of trial and error. Setting up proper database relationships was tricky, especially when trying to keep everything consistent. But honestly, the hardest part was GitHub—dealing with branch merges, fixing conflicts, and crashing the repo because we forgot to update the `.gitignore` file was a real pain. It caused plenty of frustration, but it taught us a lot about being organized and careful with version control.

While the app works well, there's definitely room to make it better. Adding features like usernames and profile pictures would give users a way to show off their identity. And a private messaging system could also make it easier for users to connect outside of posts. Finally, there's always room to polish the UI and make it even easier and more enjoyable to use.

Even though it was not always smooth sailing, this project taught us much and gave us experience creating a web app. We're proud of what we built and ready to apply what we've learned in future projects.